PROJETO DE JOGO / 01

Chees mais mais

Documento de conceito e plano de desenvolvimento • Versão 0.2 • 22/10/2026

Um roguelike tático de xadrez em um tabuleiro que começa em 5 × 5 e cresce até 8 × 5. O jogador começa com uma peça, escolhe melhorias e forma um pequeno exército para enfrentar rodadas de inimigos cada vez mais fortes. A cada cinco rodadas, um rei inimigo precisa ser derrotado por último.

Objetivo deste documento

Organizar a ideia para que o grupo consiga discutir as regras, dividir o trabalho e construir o jogo em etapas. Este material é um planejamento: não implementa o jogo nem fixa a lista de aprimoramentos que o grupo enviará depois.

Pilar	Como aparece no jogo
Identidade das peças	Peão, torre, bispo, cavalo e rei mantêm movimentos próprios; melhorias criam novas possibilidades.
Escolhas a cada rodada	Selecionar uma melhoria entre cartas sorteadas ou, nas rodadas previstas, recrutar uma peça.
Dificuldade acumulada	Dois contadores controlam a quantidade de inimigos e o orçamento de melhorias.
Leitura do combate	Tabuleiro no centro, exércitos nas laterais e capturas acima e abaixo.

Como interpretar as regras

Definido: corresponde ao pedido do grupo. Proposta: sugestão para o primeiro protótipo, sujeita a aprovação. Pendente: escolha que ainda precisa ser discutida. Essas diferenças aparecem ao longo do documento.

Requisitos do trabalho

Linguagem C + raylib; matriz para o tabuleiro; listas encadeadas para as peças; leitura e escrita de arquivo para o recorde; Makefile com suporte a Windows, Linux e macOS. Sprites e catálogo definitivo de melhorias virão depois.

Roteiro de leitura

02 Ciclo de jogo • 03 Expansão • 04 Peças e cartas • 05 Inimigos e chefes • 06 Interface e paleta • 07 Decisões • 08 Estruturas • 09 Módulos • 10 Dependências • 11 Desenvolvimento • 12 Validação.

PROJETO DE JOGO / 02

Ciclo de uma partida

Vocabulário: partida é a tentativa inteira; rodada é uma fase; turno é a vez de agir dentro do combate.

Passo	Comportamento planejado
1. Escolha inicial	Escolher peão, torre, bispo, cavalo ou rei. A rainha não aparece como escolha inicial.
2. Cartas da rodada	Atualizar a largura ativa antes das cartas. Sortear três opções válidas e escolher uma; cinco nas rodadas de chefe. A primeira rodada já inclui a primeira escolha de melhoria.
3. Recrutamento	Nas rodadas pares, pelo menos uma carta oferece ganhar uma peça. Escolhê-la abre três cartas de peças para selecionar uma.
4. Preparação inimiga	Sortear a composição permitida para a fase, transformar uma peça em rei se houver chefe e distribuir as melhorias inimigas.
5. Combate	Executar movimentos, capturas e efeitos até encerrar a rodada ou perder a partida. A ordem dos turnos ainda precisa ser decidida.
6. Resultado	Em caso de vitória, avançar a fase e manter a progressão definida para a partida. Em caso de derrota, registrar o recorde conforme a regra escolhida.

Início sem uma quarta peça inimiga

Definido: o inimigo começa com três peças normais. Proposta: iniciar a rodada 1 com pecasInimigas = 3 e melhoriasInimigas = 0. Fazer o sorteio de aumento de dificuldade somente da rodada 2 em diante, preservando esse início.

Uma escolha de recompensa por rodada

Proposta: a carta de recrutamento ocupa uma das três opções comuns ou das cinco opções de chefe; não dá uma escolha extra. Na rodada 10, por exemplo, seriam cinco cartas, sendo pelo menos uma de recrutamento. Selecionar recrutamento substitui a melhoria daquela rodada.

Progressão dentro da tentativa

Proposta: manter melhorias e aliados sobreviventes entre rodadas, reposicionar as peças antes do combate e remover capturas definitivamente daquela tentativa. Uma nova partida reinicia esse progresso e conserva apenas o recorde do arquivo. Confirmar essas regras antes da etapa de progressão.

Fluxo resumido

Escolher peça → escolher carta → preparar tabuleiro e inimigos → combater → vencer e avançar OU perder e registrar recorde. Cartas inválidas não devem consumir a escolha do jogador.

PROJETO DE JOGO / 03

Tabuleiro que cresce

Dimensões sempre descritas como largura × altura: mais colunas, mantendo cinco linhas.

Rodadas	Dimensão ativa	Casas disponíveis
1 a 5	5 × 5	25 casas; o chefe da rodada 5 ainda usa este tamanho.
6 a 10	6 × 5	30 casas; expansão após vencer o primeiro chefe.
11 a 15	7 × 5	35 casas; expansão após vencer o segundo chefe.
16 em diante	8 × 5	40 casas; limite definitivo, inclusive após os próximos chefes.

Regra definida de expansão

A cada chefe vencido, a próxima rodada ganha uma coluna, até o máximo de oito. A altura permanece em cinco. Não aumentar o tabuleiro antes da luta: a rodada 10 acontece em 6 × 5; a 11, em 7 × 5. A expansão é automática e não ocupa uma carta de melhoria.

Cálculo da largura ativa

larguraAtiva = min(8, 5 + (fase - 1) / 5), com divisão inteira e fase iniciando em 1. Ao começar uma nova tentativa, voltar a 5 × 5. Validar a fase antes de calcular as dimensões.

Matriz e coordenadas: proposta técnica

Reservar uma matriz máxima de cinco linhas e oito colunas, Peca *tabuleiro[5][8], e ativar somente as primeiras larguraAtiva colunas. Movimento, captura, sorteio de casas, IA e clique devem consultar a largura ativa. Células reservadas fora dela permanecem vazias e não podem ser usadas.

Proposta: acrescentar a coluna à direita. As coordenadas existentes permanecem iguais; o reposicionamento entre fases continua dependendo da regra de preparação. A expansão acontece antes de recrutar e posicionar peças na nova rodada. A interface centraliza o tabuleiro e ajusta o tamanho das casas para acomodar oito colunas.

Capacidade e dificuldade continuam limitadas

Crescer até 40 casas ajuda, mas não permite adicionar peças indefinidamente. Antes de criar ou recrutar, verificar casas livres e regras de posicionamento. Proposta: reservar pelo menos uma casa vazia para mobilidade e, ao atingir o limite de inimigos, aumentar melhorias se houver efeitos válidos. Se nada couber, manter a dificuldade, sem sorteio infinito.

Verificação das transições

Testar as fases 1, 5, 6, 10, 11, 15, 16, 20 e 21; confirmar que uma nova coluna fica acessível somente na fase correta, que a altura nunca muda e que o tamanho não ultrapassa 8 × 5. Conferir também cliques, capturas, seleção e sorteio nas colunas recém-abertas.

PROJETO DE JOGO / 04

Peças e aprimoramentos

O catálogo final de efeitos será fornecido pelo grupo. A estrutura deve permitir acrescentá-los sem reescrever o combate.

Peça inicial	Movimento e ação base
Peão	Uma casa para frente; duas no primeiro movimento, se o caminho estiver livre. Captura uma casa na diagonal para frente. O pedido menciona o primeiro movimento no jogo; confirmar como isso vale para peões recrutados.
Torre	Qualquer distância na horizontal ou vertical, sem atravessar outras peças.
Bispo	Qualquer distância na diagonal, sem atravessar outras peças.
Cavalo	Salta para a ponta do L: duas casas em um eixo e uma no outro. Pode saltar obstáculos.
Rei	Uma casa em qualquer direção. Ao selecionar a casa de um aliado alcançável, permanece onde está e concede a ele duas ações consecutivas na próxima jogada.

Melhorias do jogador: por tipo

Definido: sortear melhorias apenas para tipos de peça que o jogador possui; uma melhoria de peão beneficia todos os seus peões. Não repetir aprimoramentos. Proposta: guardar as melhorias por tipo durante a partida; peças recrutadas depois herdam as melhorias do seu tipo, mesmo se o grupo perder e recuperar esse tipo.

Geração das cartas

Montar pares válidos de tipo + ID de melhoria; remover pares já adquiridos e incompatíveis; sortear opções sem reposição. A regra de não repetição deve valer tanto para a mão exibida quanto para melhorias já adquiridas. Confirmar se um efeito compartilhado pode existir em tipos diferentes.

Proposta de recrutamento: sortear três tipos distintos entre os cinco iniciais. O jogador escolhe um e pode ter várias unidades do mesmo tipo. Verificar espaço disponível antes de oferecer essa carta.

Quando faltarem opções

Proposta: exibir apenas cartas válidas, mesmo que sejam menos de três ou cinco. Se não houver nenhuma, permitir avançar sem melhoria, com mensagem clara. Definir depois se haverá outra recompensa. Nunca repetir efeitos para preencher a tela nem insistir em sorteios sem fim.

Contrato do catálogo futuro

Cada melhoria precisa de: ID estável, nome, descrição, tipos compatíveis, efeito, momento de ativação, ícone e incompatibilidades. Separar melhorias do jogador por tipo e melhorias inimigas por unidade. Nenhuma lista definitiva de poderes é criada neste plano.

PROJETO DE JOGO / 05

Inimigos e chefes

A composição é refeita a cada rodada. Os contadores acumulam dificuldade, sem exigir que as mesmas peças sobrevivam entre fases.

Fases	Tipos liberados • distribuição ainda provisória
1 e 2	Somente peões.
3 e 4	Peões, bispos e torres.
5 a 9	Peões, bispos, torres e cavalos. Chefe na fase 5.
10 em diante	Adicionar rainhas, sempre sem melhorias. Chefes nas fases 10, 15, 20 etc.

Proposta para testar: pesos relativos de sorteio: peão 6, bispo 2, torre 2, cavalo 2 e rainha 1. Considerar apenas os tipos liberados. A entrada de um tipo libera sua chance de aparecer; não obriga sua presença. O rei fica reservado ao chefe. Esses pesos não são balanceamento definitivo.

Algoritmo de preparação proposto

1. A partir da rodada 2, sortear um booleano com chances iniciais iguais: true incrementa melhoriasInimigas; false incrementa pecasInimigas.
2. Sortear novamente todos os tipos permitidos, sem comparar com a rodada anterior: repetições por acaso são válidas.
3. Em rodada múltipla de cinco, escolher uma unidade e substituí-la por um rei sem melhorias. A quantidade total permanece igual.
4. Distribuir o orçamento de melhorias em pares válidos de unidade + efeito, sem repetir o mesmo efeito na mesma unidade. Excluir rainhas e rei chefe.
5. Posicionar todas as peças em casas vazias respeitando as regras de preparação.

Exemplo: rodada 1 = 3 peças e 0 melhorias; sorteio true na 2 = 3 peças e 1 melhoria; false na 3 = 4 peças e 1 melhoria. O orçamento é redistribuído do zero em cada composição, sem somar efeitos antigos novamente.

Proteções contra estados impossíveis

Propostas: se acabarem os pares válidos, manter o orçamento excedente no contador e aplicar apenas o que couber; exibir a quantidade aplicada. Se não houver espaço para mais inimigos, tentar aumentar melhorias. Se nenhuma alternativa couber, manter a dificuldade e encerrar o sorteio. Estabelecer depois um teto de unidades e regras de posicionamento.

Regra do chefe

A cada cinco rodadas, oferecer cinco cartas e impedir a captura do rei inimigo enquanto existir outro inimigo vivo. A proteção precisa existir na validação da captura, inclusive para futuros efeitos indiretos. O rei se torna capturável quando for o último; rainhas e rei chefe nunca recebem melhorias.

PROJETO DE JOGO / 06

Interface planejada

Diagrama do tabuleiro inicial 5 × 5. Direção visual definida: preto e branco com detalhes vermelhos.



Área	Informações e interação
Centro	Matriz inicial 5 × 5, expansível até 8 × 5; peça selecionada; casas de movimento e captura; indicação de turno e fase; bloqueio visível do rei chefe.
Esquerda / direita	Aliados / inimigos ativos, cada unidade com identificador e ícones de suas melhorias. Selecionar a unidade deve destacar sua casa.
Acima / abaixo	Acima: aliados que o inimigo capturou. Abaixo: inimigos que o jogador capturou. Separar o histórico da ocupação atual do tabuleiro.
Tela de cartas	Nome, tipo afetado e descrição de cada opção; três cartas comuns, cinco em chefe e três na escolha de recrutamento.

Protótipo com recursos simples

Usar formas e letras temporárias (P, T, B, C, R e D para rainha), cores de equipe e seleção por mouse. Não depender só de cor: mostrar rótulos e descrição dos ícones. Planejar rolagem nas listas laterais e nas capturas quando o conteúdo crescer.

Paleta definida: preto e branco como base; vermelho nos destaques de seleção, captura e chefe. Usar contornos, rótulos e padrões para distinguir equipes e ações, sem depender só da cor. Redimensionar e centralizar o tabuleiro ativo quando a largura aumentar.

PROJETO DE JOGO / 07

Decisões para a reunião

Resolver primeiro as regras que afetam o motor. As sugestões abaixo não substituem as escolhas do grupo.

Tema pendente	Sugestão inicial / decisão necessária
Turnos e inteligência artificial	Proposta: uma ação aliada, seguida por uma ação inimiga. O jogador escolhe qual aliado agir; a IA escolhe um movimento legal. Confirmar se o objetivo é esse ou se todas as peças agem a cada turno.
Rei aliado	Proposta: o toque usa uma ação e exige um aliado adjacente; o bônus não acumula e concede duas ações consecutivas quando esse aliado agir novamente. Definir se o buff pode afetar outro rei e como evitar ciclos de ações extras.
Regras de xeque	Proposta: captura direta, sem xeque, xeque-mate, roque ou en passant. O rei aliado não encerra sozinho a partida ao morrer. Confirmar derrota ao perder todos os aliados e vitória ao eliminar os inimigos.
Peões no limite	Definir promoção, retorno ou outro tratamento na última fileira. Proposta provisória: promoção gratuita para torre, bispo ou cavalo, sem rainha. Antes de implementar, decidir a transferência das melhorias para o novo tipo.
Primeiro avanço do peão	Definir se cada peão tem seu próprio primeiro movimento, inclusive recrutados, e se esse estado permanece entre rodadas. Proposta: estado individual que não reinicia a cada fase.
Preparação das casas	Definir reposicionamento dos sobreviventes, entrada dos recrutados, direção dos peões, área inicial e prevenção de aberturas sem saída. Proposta: aliados embaixo e inimigos em cima, sem casas compartilhadas.
Sem movimentos legais	Definir passe, empate ou derrota. Proposta: passe quando apenas um lado estiver bloqueado; empate encerra a tentativa se ambos estiverem bloqueados. Prever ciclos de movimentos também.
Recorde e duração	Definir maior fase alcançada ou concluída, fim ou modo infinito. Proposta: salvar a maior fase alcançada; entrar na 8 e perder nela registra 8. Um recorde não é um salvamento para continuar a partida.

Depois de fechar essas regras

Confirmar a distribuição de inimigos, permanência das perdas, posição da carta de recrutamento nas cinco opções do chefe, limites de população e escopo da não repetição de melhorias. Só então transformar as propostas em regras oficiais.

PROJETO DE JOGO / 08

Estruturas de dados em C

Modelo sugerido para cumprir os requisitos acadêmicos e evitar duas versões conflitantes do estado do jogo.

Estrutura	Responsabilidade
Matriz do tabuleiro	Peca *tabuleiro[5][8]; armazenar altura = 5 e larguraAtiva = 5 a 8. Índices na ordem [linha][coluna]; cada casa guarda NULL ou um ponteiro para uma peça ativa. A matriz organiza posições, mas não libera a memória das peças.
Lista de aliados e inimigos	Listas simplesmente encadeadas são donas das peças ativas. Cada nó guarda ID, tipo, equipe, posição, estado do primeiro movimento e efeitos temporários; inimigos também guardam seus IDs de melhorias.
Listas de capturas	Listas encadeadas com registros de captura: tipo, equipe, fase e IDs dos efeitos, se necessários para a interface. Guardar cópias dos dados, sem ponteiros para peças já liberadas.
Melhorias aliadas por tipo	Coleção de IDs adquiridos para cada tipo de peça. Consultada por todas as unidades daquele tipo. Não duplicar a mesma coleção dentro de cada aliado.
Estado da partida	Fase, turno, tela atual, seleção por ID, pecasInimigas, melhoriasInimigas, semente aleatória e indicação de chefe. Separar dados do recorde dos dados da tentativa.

Captura como uma operação consistente

Validar a ação e a proteção do chefe; criar o registro da captura; retirar a vítima da matriz e de sua lista; limpar seleções ou referências à vítima; liberar sua memória; mover o atacante e atualizar sua posição. Se a alocação do registro falhar, tratar o erro sem deixar o tabuleiro parcialmente atualizado.

Invariantes que o grupo deve preservar

Cada peça ativa aparece uma vez em sua lista e uma vez na matriz; posição na peça e casa da matriz concordam; nenhuma casa tem duas unidades; peça capturada não permanece ativa; toda memória alocada tem um responsável por liberá-la. Ao sortear nova rodada, limpar inimigos anteriores antes de criar os novos.

Arquivo de recorde: sugestão de formato

O pedido cita podio.tsx. Recomenda-se confirmar a intenção e usar podio.txt: arquivo simples com versão e maior_fase_alcancada. Em C, ler e escrever por stdio.h; validar números, tratar arquivo ausente ou inválido e informar falha de gravação. Não gravar ponteiros ou estruturas cruas de memória.

Proposta para o histórico visual: capturas acumuladas da tentativa, com a fase indicada. Limpar listas e coleções ao iniciar outra tentativa; manter o arquivo do recorde.

Organização e compilação

Separar regras, apresentação e persistência para permitir trabalho simultâneo entre colegas sem misturar responsabilidades.

Módulo sugerido	Conteúdo
main.c / jogo.c e .h	Inicialização da raylib, loop e máquina de estados: menu, escolha inicial, cartas, preparação, combate e resultado.
tabuleiro.c / pecas.c	Matriz, listas, criação e remoção; geração e validação de movimentos; captura. Funções de regra sem chamadas de desenho.
melhorias.c	Catálogo por ID, elegibilidade, sorteio sem repetição e aplicação de efeitos nos momentos definidos.
rodadas.c / ia.c	Contadores, tipos por fase, montagem dos inimigos, chefes e escolha de ações legais da IA.
interface.c / recursos.c	Tabuleiro, painéis, cartas, entrada e carregamento dos futuros sprites. A interface pede ações ao motor, sem editar posições diretamente.
persistencia.c	Carregar e atualizar o recorde; testar ausência, conteúdo inválido e erro de escrita.

Organização proposta: src/ para fontes; include/ para cabeçalhos; assets/ para sprites e ícones; tests/ para verificações do motor; docs/ para decisões; build/ para objetos; bin/ para executáveis. A definição dos contratos de funções vem antes da implementação separada por colegas.

Makefile para três sistemas

Planejar alvos make (compilar), make run (executar), make clean (limpar gerados) e make test (testar regras sem interface). Permitir configurar compilador, opções e caminho da raylib; compilar e vincular localmente em cada sistema. Um único executável não atende aos três sistemas.

Windows: documentar um ambiente consistente, como MSYS2/MinGW com GNU Make e raylib da mesma arquitetura. Linux: documentar compilador C, GNU Make e instalação da raylib. macOS: documentar Clang, Make e raylib. Validar os parâmetros de vínculo em cada máquina; não presumir que as bibliotecas do Windows servem nos demais.

Caminhos e documentação

Definir o diretório de trabalho de make run, o caminho dos assets e uma pasta gravável para o recorde. O alvo clean deve preservar podio.txt. No README, registrar pré-requisitos, comandos por sistema e limitações conhecidas. O menu existente em src/main.c pode ser aproveitado como ponto de entrada; a opção Continuar exigirá um salvamento de partida separado se for mantida.

Dependências e ambiente

Verificação local em 22/10/2026. As ferramentas necessárias já estavam instaladas; nenhum pacote adicional foi necessário.

Ferramenta	Estado local / função
MSYS2 / MinGW64	Ambiente Windows x86_64 em C:/msys64/mingw64. Usar ferramentas e bibliotecas da mesma arquitetura.
GCC 16.1.0	Compilador C disponível. Compilação verificada com -std=c99 -Wall -Wextra.
GNU Make 4.4.1	Disponível como mingw32-make. Executará o Makefile quando ele for implementado pelo grupo.
raylib 5.5	Cabeçalho, biblioteca e DLL presentes. O pkg-config informa 5.5.0. Não foi feita atualização de versão.
pkg-config / Git	pkg-config encontra os caminhos da raylib; Git está disponível para controle de versão.

Validação realizada neste computador

O menu existente em src/main.c compilou com sucesso. Um programa mínimo também compilou, vinculou à raylib e executou ColorToInt com resultado esperado. A janela e o funcionamento gráfico não foram testados nesta revisão. Linux e macOS ainda precisam ser verificados nas máquinas do grupo.

Preparar outras máquinas

Windows / MSYS2 MinGW64: instalar os pacotes mingw-w64-x86_64-gcc, mingw-w64-x86_64-make, mingw-w64-x86_64-raylib e mingw-w64-x86_64-pkgconf com pacman -S --needed. O comando completo está em docs/DEPENDENCIAS.md.

Linux: preparar compilador C, Make, pkg-config e a raylib, com bibliotecas gráficas e de áudio requeridas pela distribuição. Para Ubuntu/Debian, o guia complementar registra os pacotes e um caminho de compilação da raylib 5.5. macOS: instalar as Command Line Tools e, com Homebrew disponível, executar brew install raylib pkg-config. Combinar a versão usada pelo grupo; o gerenciador pode oferecer outra versão.

O que faz parte do jogo

C, biblioteca padrão e raylib atendem ao escopo atual. Python e as bibliotecas de PDF usadas para editar este documento não são dependências do jogo. Não é necessário instalá-las nas máquinas dos colegas para compilar ou jogar.

Referências oficiais e instruções completas

MSYS2: gerenciamento de pacotes • raylib: GNU/Linux • raylib: macOS. Ver comandos reproduzíveis e resultado da verificação em docs/DEPENDENCIAS.md, entregue junto ao plano.

Plano de desenvolvimento

Etapas orientadas a entregas verificáveis. A duração depende do tamanho do grupo e das horas disponíveis.

Etapa	Entrega e critério de conclusão
0 • Fechar regras	Registrar as decisões da página 7, desenhar o fluxo e combinar structs e funções públicas. Conclusão: todos conseguem descrever um turno sem ambiguidades.
1 • Base executável	Configurar C, raylib e Makefile; aproveitar o menu; desenhar a matriz ativa 5 × 5 com letras e prever expansão até 8 × 5. Conclusão: compila, abre e fecha sem erros nos ambientes disponíveis.
2 • Motor e memória	Implementar matriz e listas, movimentos, bloqueios e capturas; usar inimigos fixos. Conclusão: ações válidas funcionam e ilegais não alteram o estado.
3 • Combate completo	Adicionar ordem de turnos aprovada, IA simples, efeito base do rei, vitória, derrota e bloqueios. Conclusão: concluir ou perder uma rodada sem intervenção no código.
4 • Progressão	Implementar cartas, recrutamento, melhorias por tipo, contadores inimigos e expansão nas fases 6, 11 e 16. Usar efeitos temporários de teste aprovados pelo grupo. Conclusão: avançar rodadas com estado coerente e sem opções repetidas.
5 • Chefes e recorde	Cinco cartas em chefes, rei protegido até ser o último, rainha sem melhorias, leitura e escrita do recorde. Conclusão: jogar até a fase 10 e reabrir o jogo com o recorde preservado.
6 • Conteúdo e entrega	Integrar catálogo e sprites do grupo, ajustar interface e balanceamento, testar os três sistemas e concluir README. Conclusão: outra pessoa compila e joga seguindo apenas a documentação.

Divisão sugerida em frentes

Motor e dados: matriz, listas, movimentos e memória. Progressão: cartas, catálogo, geração inimiga e chefes. Interface e arte: telas, interação, painéis e sprites. Integração: Makefile, recorde, testes e documentação. Adaptar a distribuição ao número de colegas; uma pessoa pode assumir mais de uma frente.

Integração do grupo

Antes de dividir arquivos, combinar nomes de tipos, funções e responsabilidade da memória. Integrar uma pequena entrega por vez, revisar em dupla e manter o ramo principal compilando. Cada etapa depende da anterior; a arte pode avançar em paralelo depois de acordados tamanho e identificação dos recursos.

Primeiro marco jogável

Uma rodada com peças temporárias, movimentos e capturas corretos, IA simples e vitória/derrota. Só depois acrescentar cartas, progressão e chefes. Não é necessário esperar os sprites finais para validar a ideia.

Validação e próximos passos

Checklist de aceite para o grupo usar durante o desenvolvimento; estes testes ainda serão implementados.

Área	Cenários essenciais
Movimento	Linhas 0 a 4 e colunas 0 a larguraAtiva - 1; células inativas inacessíveis; torre e bispo bloqueados; salto do cavalo; peão só captura na diagonal; avanço duplo com caminho livre; rei não sobrepõe um aliado ao aplicar o buff.
Expansão	Conferir 5 × 5, 6 × 5, 7 × 5 e 8 × 5 nas fases 1, 6, 11 e 16; testar limite após fase 20, reinício em 5 × 5 e bloqueio de colunas inativas.
Estado e memória	Capturar retira a peça de matriz e lista; remoção do primeiro, do meio e do último nó; reiniciar tentativa limpa dados; falha de alocação não deixa estado parcial.
Cartas e recrutamento	Primeira escolha fornece uma melhoria; apenas tipos possuídos são elegíveis; melhoria vale para todos do tipo; opções não repetem; rodadas pares incluem recrutamento; catálogo esgotado e tabuleiro cheio não travam.
Geração e dificuldade	Primeira rodada com três peças normais; só um contador aumenta por transição; composição é ressorteada; tipos respeitam fase mínima; efeitos não repetem na mesma unidade; orçamento excedente tem saída finita.
Chefes e efeitos	Fases 5 e 10 com cinco cartas; rei substitui uma unidade; chefe não é capturável antes dos demais, nem por efeitos; fica vulnerável ao ser o último; rainha e chefe sem melhorias.
Persistência e sistemas	Arquivo ausente, inválido e sem permissão; recorde não diminui; recorde preservado após reinício e clean; compilar e jogar em Windows, Linux e macOS com instruções reproduzíveis.

Testar o acaso sem perder a reprodução

Usar uma semente fixa durante a depuração e registrá-la ao investigar uma falha. Validar propriedades do sorteio e casos limite; não exigir que rodadas consecutivas sejam sempre diferentes. Fazer sessões de jogo para avaliar dificuldade, duração, utilidade das peças e facilidade de leitura da interface.

Pauta para a próxima conversa do grupo

1. Fechar ordem dos turnos, derrota, xeque, rei aliado e tratamento dos peões.
2. Aprovar ou ajustar a distribuição de inimigos, posicionamento e limites de peças.
3. Definir o catálogo: IDs, efeitos, incompatibilidades e ícones.
4. Confirmar formato e significado do recorde; decidir se haverá Continuar.
5. Distribuir responsáveis e combinar a data do primeiro marco jogável.

Escopo desta entrega

Documento de conceito, propostas explícitas e roteiro de implementação. O código existente foi apenas consultado; o jogo, o catálogo definitivo de melhorias, os sprites e o Makefile ficam para as etapas que vocês desenvolverão.